



Protocol Audit Report

Version 1.0

Dakray Security

February 3, 2026

Puppy Raffle Audit Report

Dakray Security

February 3, 2026

Prepared by: [Dakray Security] Lead Security Researcher: - Dakota Rogers

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - [H-01] Unbounded Gas Consumption in `enterRaffle` Due to Duplicate Check
 - [H-02] `getActivePlayerIndex` Returns 0 for Non-Existent Players (Index Confusion)
 - [H-03] Reentrancy in `refund` Allows Draining Raffle Balance
 - [H-04] Weak Randomness in `selectWinner` Enables Winner Manipulation (Including Front-Running)
 - [H-05] `totalFees uint64` Overflow Causes Fee Accounting Loss
 - [H-06] `refund()` Leaves `address(0)` Holes Which Can Break `selectWinner()`
 - [H-07] `enterRaffle([])` Triggers Underflow DoS in Solidity <0.8.0

- [M-01] `withdrawFees` Strict Equality Check Allows Permanent DoS (Forced ETH)
- [M-02] Off-By-One Error in Rarity Distribution
- [L-01] `selectWinner` Uses Inclusive End-Time Check ($\geq$)
- [L-02] `enterRaffle` Payment Check Can Overflow (Solidity $<0.8.0$)
- [I-01] Unused Internal Function `_isActivePlayer` Wastes Gas
- [I-02] Solidity Pragma Should Be Specific, Not Wide
- [I-03] Using An Outdated Version of Solidity Is Not Recommended
- [I-04] Missing Checks for `address(0)` When Assigning Values to Address State Variables
- [I-05] `PuppyRaffle::selectWinner` Does Not Follow CEI Pattern
- [I-06] Use of “Magic” Numbers Is Discouraged
- [G-01] Unchanged State Variables Should Be Declared Constant or Immutable
- [G-02] Storage Variables in a Loop Should Be Cached

Protocol Summary

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the `enterRaffle` function with the following parameters:
 1. `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.
2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & `value` if they call the `refund` function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a `feeAddress` to take a cut of the `value`, and the rest of the funds will be sent to the winner of the puppy.

Disclaimer

The Dakray Security team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

Likelihood	High Impact	Medium Impact	Low Impact
High	H	H/M	M
Medium	H/M	M	M/L
Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

```
1 ./src/  
2 |-- PuppyRaffle.sol
```

- Commit Hash: 22bbbb2c47f3f2b78c1b134590baf41383fd354f
- Contest: Cyfrin CodeHawks First Flight #2: Puppy Raffle
- Contest dates: Oct 25 2023 - Nov 01 2023

Roles

Owner: - deployer of the protocol - can change the wallet address to which fees are sent through `changeFeeAddress`

Player: - can enter the raffle with `enterRaffle` - can refund value through `refund`

Executive Summary

This was a contest on Cyfrin Codehawks.

Issues found

Severity	Number of Issues found
High	7
Medium	2
Low	2
Info	6
Gas	2

Findings

[H-01] Unbounded Gas Consumption in enterRaffle Due to Duplicate Check

Description:

The `PuppyRaffle::enterRaffle` function uses nested loops to check for duplicate players, creating $O(n^2)$ time complexity. Gas costs grow exponentially as the array size increases, eventually exceeding block gas limits and preventing anyone from entering.

```
1  for (uint256 i = 0; i < players.length - 1; i++) {
2      for (uint256 j = i + 1; j < players.length; j++) {
3          require(players[i] != players[j], "PuppyRaffle: Duplicate
           player");
4      }
5  }
```

Impact:

Complete denial of service: raffle becomes unusable for all participants once entry exceeds the block gas limit.

Proof of Concept:

POC

Place the following into `test/PuppyRaffleTest.t.sol`:

```
1  function test_denialOfServiceViaGasCosts() public {
2      // Step 1: Baseline - enter with 1 player
3      address[] memory player = new address[](1);
4      player[0] = address(10);
5      vm.deal(address(10), entranceFee);
6
7      vm.prank(address(10));
8      uint256 gasStart = gasleft();
```

```
9 puppyRaffle.enterRaffle{value: entranceFee}(player);
10 uint256 gasUsed1 = gasStart - gasleft();
11
12 // Step 2: Enter 99 more players (total 100)
13 address[] memory players100 = new address[] (99);
14 for (uint256 i = 0; i < 99; i++) {
15     players100[i] = address(uint160(i + 100));
16 }
17
18 vm.deal(address(100), entranceFee * 99);
19 vm.prank(address(100));
20 puppyRaffle.enterRaffle{value: entranceFee * 99}(players100);
21
22 // Measure gas with 100 players in array
23 address player101 = address(1000);
24 address[] memory playerArray101 = new address[] (1);
25 playerArray101[0] = player101;
26
27 vm.deal(player101, entranceFee);
28 vm.prank(player101);
29 gasStart = gasleft();
30 puppyRaffle.enterRaffle{value: entranceFee}(playerArray101);
31 uint256 gasUsed100 = gasStart - gasleft();
32
33 // Step 3: Enter 699 more players (total 800)
34 address[] memory players800 = new address[] (699);
35 for (uint256 i = 0; i < 699; i++) {
36     players800[i] = address(uint160(i + 2000));
37 }
38
39 vm.deal(address(2000), entranceFee * 699);
40 vm.prank(address(2000));
41 puppyRaffle.enterRaffle{value: entranceFee * 699}(players800);
42
43 // Measure gas with 800 players in array
44 address player801 = address(9000);
45 address[] memory playerArray801 = new address[] (1);
46 playerArray801[0] = player801;
47
48 vm.deal(player801, entranceFee);
49 vm.prank(player801);
50 gasStart = gasleft();
51 puppyRaffle.enterRaffle{value: entranceFee}(playerArray801);
52 uint256 gasUsed800 = gasStart - gasleft();
53
54 // Show progression
55 emit log("=== Gas Cost Analysis ===");
56 emit log_named_uint("Gas with 1 player", gasUsed1);
57 emit log_named_uint("Gas with 100 players", gasUsed100);
58 emit log_named_uint("Gas with 800 players", gasUsed800);
59 emit log("Block gas limit: 30000000");
```

```
60
61     assertGt(gasUsed800, 30_000_000, "DoS: 800 players exceeds block
62     gas limit");
63 }
```

Run: `forge test --match-test test_denialOfServiceViaGasCosts -vv`

Recommended Mitigation:

Remove the nested loop and use a mapping-based membership check (per-raffle id mapping is a common pattern).

[H-02] `getActivePlayerIndex` Returns 0 for Non-Existent Players (Index Confusion)

Description:

`getActivePlayerIndex` returns 0 both when a player is not found and when a player is legitimately at index 0.

```
1 function getActivePlayerIndex(address player) external view returns (
    uint256) {
2     for (uint256 i = 0; i < players.length; i++) {
3         if (players[i] == player) {
4             return i;
5         }
6     }
7     return 0;
8 }
```

Impact:

Index ambiguity can break integrations and UIs, causing users who are not entered to appear “entered at index 0”.

Proof of Concept:

POC

Place the following into `test/PuppyRaffleTest.t.sol`:

```
1 function test_getActivePlayerIndexReturnsZeroForNonExistentPlayers()
    public {
2     // Setup: Three players enter
3     address alice = address(1);
4     address bob = address(2);
5     address charlie = address(3);
6
7     address[] memory players = new address[](3);
```

```
8     players[0] = alice;
9     players[1] = bob;
10    players[2] = charlie;
11
12    vm.deal(alice, entranceFee * 3);
13    vm.prank(alice);
14    puppyRaffle.enterRaffle{value: entranceFee * 3}(players);
15
16    // Query legitimate player at index 0
17    uint256 aliceIndex = puppyRaffle.getActivePlayerIndex(alice);
18
19    // Query non-existent player
20    address notInRaffle = address(99);
21    uint256 notInRaffleIndex = puppyRaffle.getActivePlayerIndex(
22        notInRaffle);
23
24    // Query another non-existent player
25    address alsoNotInRaffle = address(100);
26    uint256 alsoNotInRaffleIndex = puppyRaffle.getActivePlayerIndex(
27        alsoNotInRaffle);
28
29    emit log("=== Index Ambiguity Demonstration ===");
30    emit log_named_address("Player at index 0", puppyRaffle.players(0))
31        ;
32    emit log_named_uint("Alice (at index 0) returns", aliceIndex);
33    emit log_named_uint("Non-participant (address 99) returns",
34        notInRaffleIndex);
35    emit log_named_uint("Non-participant (address 100) returns",
36        alsoNotInRaffleIndex);
37
38    // All return 0 - impossible to distinguish!
39    assertEq(aliceIndex, 0);
40    assertEq(notInRaffleIndex, 0);
41    assertEq(alsoNotInRaffleIndex, 0);
42 }
```

Run: `forge test --match-test test_getActivePlayerIndexReturnsZeroForNonExistentPla`
`-vv`

Recommended Mitigation:

Revert when a player is not found (or return `(bool found, uint256 index)`).

[H-03] Reentrancy in `refund` Allows Draining Raffle Balance

Description:

`refund` violates Checks-Effects-Interactions by sending ETH before updating state.


```
1 payable(msg.sender).sendValue(entranceFee);
2 players[playerIndex] = address(0);
```

Impact:

An attacker can reenter `refund` and drain the contract balance.

Proof of Concept:

POC

Place the following into `test/PuppyRaffleTest.t.sol`:

```
1 function test_reentrancyRefund() public {
2     // Setup: Attacker and 3 normal players enter
3     ReentrancyAttacker attacker = new ReentrancyAttacker(puppyRaffle);
4
5     address[] memory players = new address[](4);
6     players[0] = address(attacker);
7     players[1] = playerTwo;
8     players[2] = playerThree;
9     players[3] = playerFour;
10
11     vm.deal(address(this), entranceFee * 4);
12     puppyRaffle.enterRaffle{value: entranceFee * 4}(players);
13
14     uint256 contractBalanceBefore = address(puppyRaffle).balance;
15     uint256 attackerBalanceBefore = address(attacker).balance;
16
17     emit log("=== Before Attack ===");
18     emit log_named_uint("Contract balance", contractBalanceBefore);
19     emit log_named_uint("Attacker balance", attackerBalanceBefore);
20     emit log_named_uint("Attacker paid", entranceFee);
21
22     // Execute reentrancy attack
23     attacker.attack();
24
25     uint256 contractBalanceAfter = address(puppyRaffle).balance;
26     uint256 attackerBalanceAfter = address(attacker).balance;
27
28     emit log("");
29     emit log("=== After Attack ===");
30     emit log_named_uint("Contract balance", contractBalanceAfter);
31     emit log_named_uint("Attacker balance", attackerBalanceAfter);
32     emit log_named_uint("Attacker profit", attackerBalanceAfter -
33         attackerBalanceBefore);
34
35     // Attacker should have stolen more than they paid
36     assertGt(attackerBalanceAfter - attackerBalanceBefore, entranceFee);
37 }
```

Run: `forge test --match-test test_reentrancyRefund -vv`

Recommended Mitigation:

Update state before external calls (CEI) and/or use `nonReentrant`.

[H-04] Weak Randomness in selectWinner Enables Winner Manipulation (Including Front-Running)**Description:**

`selectWinner` uses predictable on-chain values for randomness.

```
1 uint256 winnerIndex =  
2   uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,  
    block.difficulty))) % players.length;
```

Impact:

Attackers can bias or control winner selection and rarity outcomes.

Proof of Concept:

POC

Run: `-forge test --match-test test_weakPRNG -vv -forge test --match-test test_frontRunningSelectWinner -vv`

Recommended Mitigation:

Use a secure randomness mechanism (VRF or commit-reveal).

[H-05] totalFees uint64 Overflow Causes Fee Accounting Loss**Description:**

`totalFees` is a `uint64`, and `uint64(fee)` downcast can overflow in Solidity 0.7.6.

```
1 totalFees = totalFees + uint64(fee);
```

Impact:

Protocol loses fee revenue due to silent overflow/wraparound.

Proof of Concept:

POC

Run: `forge test --match-test test_totalFeesOverflow -vv`

Recommended Mitigation:

Use `uint256 totalFees` and avoid unsafe casts.

[H-06] `refund()` Leaves `address(0)` Holes Which Can Break `selectWinner()`**Description:**

`refund` replaces a slot with `address(0)` without shrinking the array, but `selectWinner` uses `players.length` for accounting and winner selection.

```
1 players[playerIndex] = address(0);
2 uint256 totalAmountCollected = players.length * entranceFee;
3 address winner = players[winnerIndex];
```

Impact:

`selectWinner` can revert due to insufficient balance after refunds, and can revert when minting to `address(0)` if `winner == address(0)`. Also enables mempool/TOD griefing by front-running `selectWinner` with `refund`.

Proof of Concept:

POC

Place the following into `test/PuppyRaffleTest.t.sol`:

```
1 function test_selectWinnerRevertsAfterRefund() public playersEntered {
2     vm.prank(playerFour);
3     puppyRaffle.refund(3);
4
5     vm.warp(block.timestamp + duration + 1);
6     vm.roll(block.number + 1);
7
8     vm.expectRevert("PuppyRaffle: Failed to send prize pool to winner")
9         ;
10    puppyRaffle.selectWinner();
11 }
12 function test_selectWinnerRevertsMintToZeroAfterAllRefunds() public
13     playersEntered {
14     vm.prank(playerOne);
15     puppyRaffle.refund(0);
16     vm.prank(playerTwo);
```

```
16     puppyRaffle.refund(1);
17     vm.prank(playerThree);
18     puppyRaffle.refund(2);
19     vm.prank(playerFour);
20     puppyRaffle.refund(3);
21
22     vm.deal(address(puppyRaffle), 10 ether);
23
24     vm.warp(block.timestamp + duration + 1);
25     vm.roll(block.number + 1);
26
27     vm.expectRevert("ERC721: mint to the zero address");
28     puppyRaffle.selectWinner();
29 }
```

```
Run: - forge test --match-test test_selectWinnerRevertsAfterRefund -vv -
forge test --match-test test_selectWinnerRevertsMintToZeroAfterAllRefunds
-vvv
```

Recommended Mitigation:

Remove refunded players (swap-and-pop), or track active players separately.

[H-07] enterRaffle([]) Triggers Underflow DoS in Solidity <0.8.0**Description:**

Calling `enterRaffle` with an empty array can underflow `players.length - 1` in the duplicate-check loop bounds.

```
1  for (uint256 i = 0; i < players.length - 1; i++) {
2      // ...
3  }
```

Impact:

Unexpected revert path / DoS condition on a core entrypoint.

Proof of Concept:

POC

Place the following into `test/PuppyRaffleTest.t.sol`:

```
1  function test_enterRaffleEmptyArrayUnderflow() public {
2      address[] memory empty = new address[](0);
3      bytes memory data = abi.encodeWithSelector(puppyRaffle.enterRaffle.selector, empty);
```

```
4
5     (bool ok,) = address(puppyRaffle).call{value: 0, gas: 200_000}(data
6         );
7     assertEq(ok, false);
8 }
```

Run: `forge test --match-test test_enterRaffleEmptyArrayUnderflow -vv`

Recommended Mitigation:

Require non-empty input and/or use safe loop bounds (`i + 1 < players.length`).

[M-01] withdrawFees Strict Equality Check Allows Permanent DoS (Forced ETH)

Description:

Strict equality against `address(this).balance` can be broken by forced ETH.

```
1 require(address(this).balance == uint256(totalFees), "PuppyRaffle:
   There are currently players active!");
```

Impact:

Fees can be permanently locked.

Proof of Concept:

POC

Place the following into `test/PuppyRaffleTest.t.sol`:

```
1 function test_withdrawFeesDoS() public playersEntered {
2     vm.warp(block.timestamp + duration + 1);
3     vm.roll(block.number + 1);
4
5     puppyRaffle.selectWinner();
6
7     uint256 totalFees = puppyRaffle.totalFees();
8     uint256 contractBalance = address(puppyRaffle).balance;
9
10    emit log("=== Before Attack ===");
11    emit log_named_uint("Total fees", totalFees);
12    emit log_named_uint("Contract balance", contractBalance);
13    emit log_named_string("Can withdraw", contractBalance == totalFees
14        ? "Yes" : "No");
15
16    SelfDestructAttacker attacker = new SelfDestructAttacker();
17    vm.deal(address(attacker), 1);
18    attacker.attack(address(puppyRaffle));
19 }
```

```
18
19     contractBalance = address(puppyRaffle).balance;
20
21     emit log("");
22     emit log("=== After Attack (1 wei forced in) ===");
23     emit log_named_uint("Total fees", totalFees);
24     emit log_named_uint("Contract balance", contractBalance);
25     emit log_named_uint("Difference", contractBalance - totalFees);
26
27     vm.expectRevert("PuppyRaffle: There are currently players active!")
28         ;
29     puppyRaffle.withdrawFees();
30 }
```

Run: `forge test --match-test test_withdrawFeesDoS -vv`

Recommended Mitigation:

Avoid strict equality against `address(this).balance` (use `>=` or an invariant based on active players).

[M-02] Off-By-One Error in Rarity Distribution

Description:

`rarity % 100` yields 0..99, but `<=` boundaries allocate 71 outcomes to “common” and only 4 to “legendary”.

```
1 if (rarity <= COMMON_RARITY) { /* ... */ }
```

Impact:

Drop rates do not match intended distribution.

Proof of Concept:

POC

Place the following into `test/PuppyRaffleTest.t.sol`:

```
1 function test_rarityDistribution() public {
2     uint256 commonCount = 0;
3     uint256 rareCount = 0;
4     uint256 legendaryCount = 0;
5
6     for (uint256 rarity = 0; rarity < 100; rarity++) {
7         if (rarity <= puppyRaffle.COMMON_RARITY()) {
8             commonCount++;
9         }
10    }
```

```
9      } else if (rarity <= puppyRaffle.COMMON_RARITY() + puppyRaffle.  
10         RARE_RARITY()) {  
11         rareCount++;  
12     } else {  
13         legendaryCount++;  
14     }  
15  
16     assertEq(commonCount, 71);  
17     assertEq(rareCount, 25);  
18     assertEq(legendaryCount, 4);  
19 }
```

Run: `forge test --match-test test_rarityDistribution -vv`

Recommended Mitigation:

Use < boundaries (0..69 common, 70..94 rare, 95..99 legendary).

[L-01] selectWinner Uses Inclusive End-Time Check (>=)

Description:

`selectWinner` allows finalization at the exact boundary timestamp due to >=.

```
1 require(block.timestamp >= raffleStartTime + raffleDuration, "  
   PuppyRaffle: Raffle not over");
```

Impact:

Edge-case boundary behavior (earliest-possible finalization).

Proof of Concept:

POC

Place the following into `test/PuppyRaffleTest.t.sol`:

```
1 function test_selectWinnerAllowedAtExactEndTime() public playersEntered  
2 {  
3     uint256 endTime = puppyRaffle.raffleStartTime() + puppyRaffle.  
4         raffleDuration();  
5     vm.warp(endTime);  
6     vm.roll(block.number + 1);  
7     puppyRaffle.selectWinner();  
8 }
```

```
Run: forge test --match-test test_selectWinnerAllowedAtExactEndTime -vv
```

Recommended Mitigation:

Use > if strict “after end time” behavior is desired.

[L-02] enterRaffle Payment Check Can Overflow (Solidity <0.8.0)**Description:**

`entranceFee * newPlayers.length` can overflow in Solidity 0.7.6.

```
1 require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle:
   Must send enough to enter raffle");
```

Impact:

In overflow edge-cases, entrants can underpay while passing the equality check.

Proof of Concept:

POC

Place the following into `test/PuppyRaffleTest.t.sol`:

```
1 function test_enterRafflePaymentOverflowAllowsFreeEntry() public {
2     uint256 hugeEntranceFee = type(uint256).max / 2 + 1;
3     puppyRaffle = new PuppyRaffle(hugeEntranceFee, feeAddress, duration
4         );
5     address[] memory players = new address[](2);
6     players[0] = playerOne;
7     players[1] = playerTwo;
8
9     puppyRaffle.enterRaffle{value: 0}(players);
10    assertEq(puppyRaffle.players(0), playerOne);
11    assertEq(puppyRaffle.players(1), playerTwo);
12 }
```

```
Run: forge test --match-test test_enterRafflePaymentOverflowAllowsFreeEntry -vv
```

Recommended Mitigation:

Add an overflow guard or use SafeMath.

[I-01] Unused Internal Function `_isActivePlayer` Wastes Gas**Description:**

`_isActivePlayer` is never used.

Recommended Mitigation:

Remove unused code.

[I-02] Solidity Pragma Should Be Specific, Not Wide**Description:**

Wide pragma `^0.7.6` can lead to inconsistent builds.

Recommended Mitigation:

Pin to `pragma solidity 0.7.6;`

[I-03] Using An Outdated Version of Solidity Is Not Recommended**Description:**

Solidity 0.7.6 lacks built-in overflow checks and newer safety improvements.

Recommended Mitigation:

Upgrade to 0.8.x when feasible.

[I-04] Missing Checks for `address(0)` When Assigning Values to Address State Variables**Description:**

`changeFeeAddress` allows setting `feeAddress` to `address(0)`.

Recommended Mitigation:

Add a zero-address check.

[I-05] PuppyRaffle::selectWinner Does Not Follow CEI Pattern**Description:**

`selectWinner` makes external calls (ETH transfer + `_safeMint`) after some state changes; CEI is recommended for safety.

Recommended Mitigation:

Complete all effects before interactions where possible.

[I-06] Use of “Magic” Numbers Is Discouraged**Description:**

Fee/prize percentages use literals (80, 20, 100).

Recommended Mitigation:

Use named constants.

[G-01] Unchanged State Variables Should Be Declared Constant or Immutable**Description:**

Prefer `immutable/constant` for values that do not change.

Recommended Mitigation:

Mark never-changing values as `immutable/constant`.

[G-02] Storage Variables in a Loop Should Be Cached**Description:**

Cache storage reads like `players.length` in loops to reduce gas (note: H-01 recommends removing the loop entirely).

Recommended Mitigation:

Cache `players.length` before looping (or remove the $O(n^2)$ loop entirely).
